Recitation 13

Link Layer - Bridges & Exam Review

Daniel Karalnik Liam Tal

Winter Semester 2025

Table of Contents

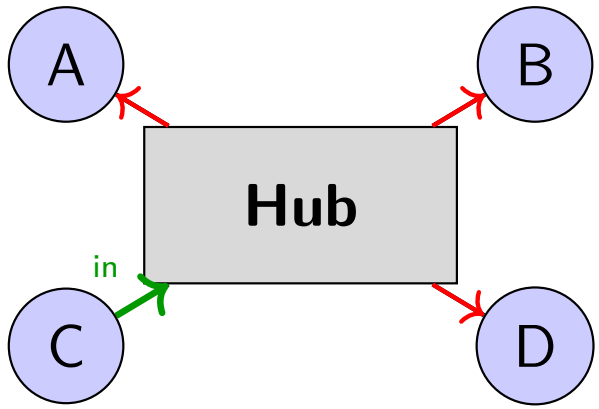
1. Link Layer - Bridges
   - Network Devices Overview
   - Bridge Table
   - Combined Topology Example
2. Exam Questions
   - Bridge & ARP Questions
   - DHCP Questions
   - TCP Questions
   - Sliding Window Protocols
   - Transport Layer Calculations
   - Network Delay Calculations
   - HTTP Timing

Link Layer - Bridges

What is a Hub?

Hub

A **hub** is a simple network device that connects multiple devices in a LAN.



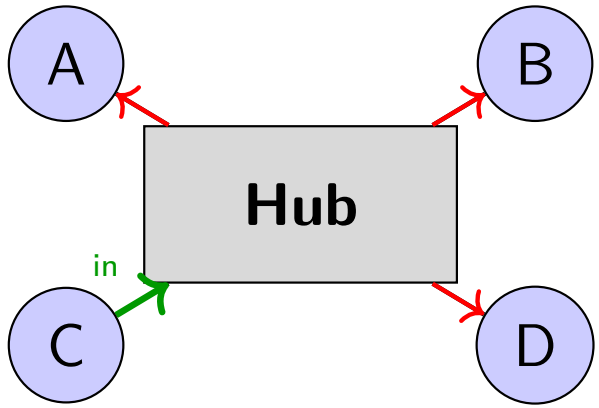
Signal from C is repeated to all other ports

What is a Hub?

Hub

Characteristics:

- Operates at **Physical Layer** (Layer 1)
- Acts as a **multi-port repeater**
- No intelligence – just amplifies signals
- Creates a single **collision domain**

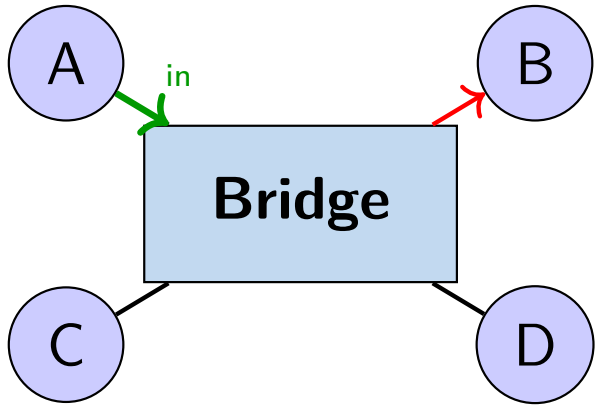


Signal from C is repeated to all other ports

What is a Bridge?

Bridge

A **bridge** is a network device that connects and filters traffic between LAN segments.



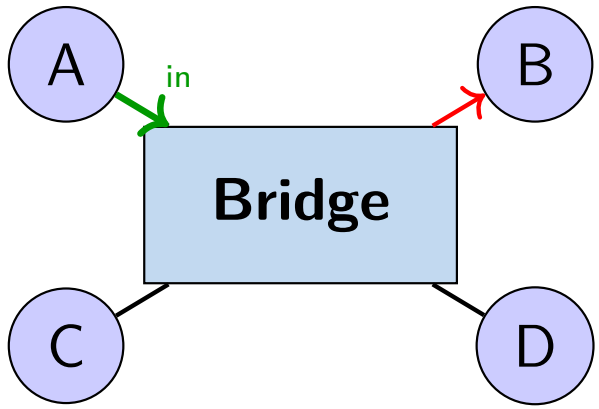
A→B: Bridge forwards to B only (selective)

What is a Bridge?

Bridge

Characteristics:

- Operates at **Link Layer** (Layer 2)
- Makes forwarding decisions based on **MAC addresses**
- **Self-learning** – builds forwarding table automatically
- Creates **separate collision domains**
- Transparent to end hosts



A→B: Bridge forwards to B only (selective)

Bridges and Switches

Terminology Note

A **switch** is essentially a **multi-port bridge** with hardware acceleration.

In modern usage, the terms are often used interchangeably for Layer 2 devices.

Bridges and Switches

Unmanaged Switch

- Plug-and-play
- No configuration needed
- Self-learning only
- No IP address required
- Basic forwarding based on MAC
- *Acts exactly like a bridge*

Managed Switch

- Requires configuration
- Has IP address for management
- Supports advanced features and configurations such as:
  - Supports VLANs
  - Port mirroring, QoS
  - SNMP monitoring
  - Access control lists (ACLs)

Bridges and Switches

Exam Note

In this course, when we say "bridge", we typically mean an **un-managed switch** – a simple Layer 2 device with self-learning and MAC-based forwarding.

Bridge Table

Bridge Table

- Saves association of MAC address to interface
- **Not an ARP table!**

Forwarding Logic

- If destination in table $\rightarrow$ send to relevant interface
- Else $\rightarrow$ sends to **all** interfaces (flooding)

Bridge Self-Learning

How Bridges Learn MAC Addresses

When a frame arrives on an interface, the bridge:

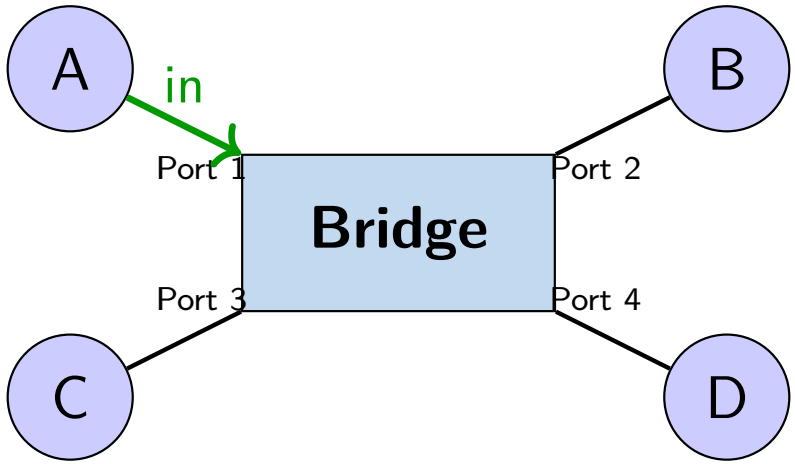
1. Reads the **source MAC address** from the frame header
2. Records the association: `Source MAC` $\rightarrow$ `Incoming Interface`
3. Stores entry with a **TTL** (Time To Live) timer

Key Insight

Bridges learn from **source addresses**, not destinations!

If host A sends a frame via port 1, the bridge now knows: "To reach A, use port 1"

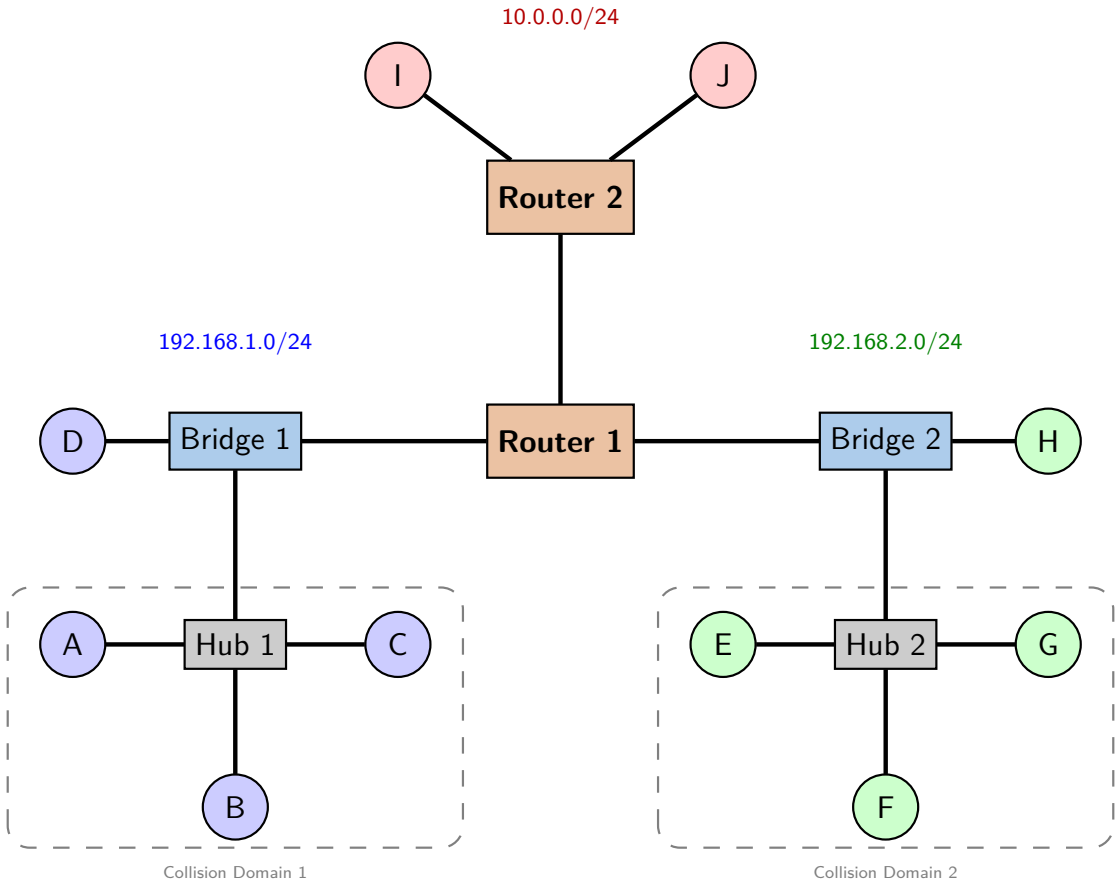
Bridge Table Example



Bridge Table After Frame

MAC Address	Interface	TTL
MAC_A	Port 1	60s

Network Topology: Routers, Bridges, and Hubs



Topology Analysis: Device Roles

Routers

- Connect **different subnets**
- Forward based on **IP addresses**
- Decrement TTL
- Separate **broadcast domains**

Bridges

- Connect **LAN segments**
- Forward based on **MAC addresses**
- Self-learning
- Separate **collision domains**

Hubs

- Connect hosts in **same segment**
- Repeat to **all ports**
- No intelligence
- Single **collision domain**

Exam Questions

Exercise 1: ARP Query Forwarding

True/False

When an ARP query arrives to a bridge the bridge always forwards the query on all the links that are connected to it **except** the link on which the query has arrived.

Solution

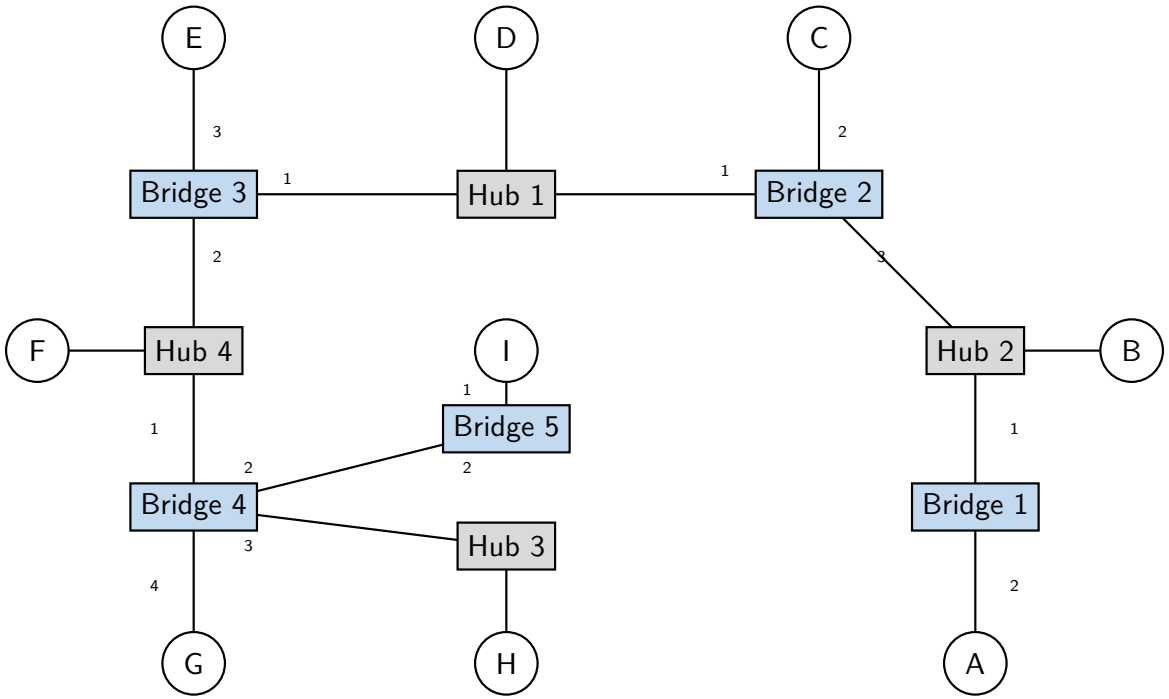
True

ARP queries are broadcast frames (destination MAC = FF:FF:FF:FF:FF:FF), and bridges always flood broadcast frames to all ports except the incoming port.

Exercise 2: LAN Topology

Problem

Consider the LAN shown below. Suppose that the forwarding table for each bridge is **empty**. Assume that all ARP tables are **empty**. For each scenario, state which hosts will see the **data packet**.



Exercise 2a: A to C

Scenario

Host A sends a data packet to Host C. Which hosts will see the data packet?

Exercise 2a: A to C

Solution

B, C

1. A sends ARP query (broadcast) $\rightarrow$ flooded everywhere
2. Bridges learn: A is on their incoming port
3. C responds with ARP reply $\rightarrow$ bridges learn C's location
4. A sends data packet to C's MAC
5. Bridge 1 knows C (port 1) $\rightarrow$ forwards to Hub 2
6. Hub 2 floods $\rightarrow$ **B sees packet**
7. Bridge 2 knows C (port 2) $\rightarrow$ forwards directly to **C**

Exercise 2b: B to F

Scenario

After the previous scenario, Host B sends a data packet to Host F. Which hosts will see the data packet?

Exercise 2b: B to F

Solution

D, F

1. B sends ARP query for F (broadcast) $\rightarrow$ bridges learn B's location
2. F responds with ARP reply $\rightarrow$ bridges learn F's location
3. B sends data packet to F's MAC address
4. Hub 2 floods to Bridge 1 and Bridge 2
5. Bridge 1: packet came from port 1, F is on port 1 $\rightarrow$ no forward
6. Bridge 2 knows F (port 1) $\rightarrow$ forwards to Hub 1
7. Hub 1 floods $\rightarrow$ **D sees packet**
8. Bridge 3 knows F (port 2) $\rightarrow$ forwards to Hub 4
9. Hub 4 floods $\rightarrow$ **F receives packet**

Exercise 2c: F to A

Scenario

After the previous scenarios, Host F sends a data packet to Host A. Which hosts will see the data packet?

Exercise 2c: F to A

Solution

A, B, D

1. F sends ARP for A (broadcast) $\rightarrow$ A responds, bridges update tables
2. F sends data to A's MAC
3. Hub 4 floods to Bridge 3 and Bridge 4
4. Bridge 3 knows A (port 1) $\rightarrow$ forwards to Hub 1
5. Hub 1 floods $\rightarrow$ **D sees packet**
6. Bridge 2 knows A (port 3) $\rightarrow$ forwards to Hub 2
7. Hub 2 floods $\rightarrow$ **B sees packet**
8. Bridge 1 knows A (port 2) $\rightarrow$ forwards to **A**

Exercise 2d: H to G

Scenario

After all previous scenarios, Host H sends a data packet to Host G. Which hosts will see the data packet?

Exercise 2d: H to G

Solution

G only

1. H sends ARP for G (broadcast) $\rightarrow$ Bridge 4 learns H (port 3)
2. G responds with ARP reply $\rightarrow$ Bridge 4 learns G (port 4)
3. H sends data to G's MAC
4. Hub 3 floods to Bridge 4 (port 3)
5. Bridge 4 knows G (port 4) $\rightarrow$ forwards directly to **G**
6. No hub in path to G $\rightarrow$ no other hosts see the packet

Key insight: Bridge learning eliminates unnecessary flooding!

Exercise 3: DHCP and MAC Address

True/False

A user can get the MAC address of its first hop router by using a DHCP server.

Solution

False
DHCP provides:

- IP address
- Subnet mask
- Default gateway **IP address**
- DNS server address

Exercise 4: TCP Properties

Multiple Choice

The question refers to TCP. What is the correct answer?

1. In congestion control, when the timer expires at the sender, the threshold value never increases.
2. The value of the timer might change throughout the duration of the connection.
3. TCP always returns an ACK for the most recent correctly received segment.
4. All of the above
5. None of the above

Exercise 4: TCP Properties - Solution

Solution

Answer: 2

The value of the timer might change throughout the duration of the connection.

Analysis

- **Option 1:** False – threshold is set to cwnd/2, which varies
- **Option 2:** **True** – TCP adjusts RTO (Retransmission Timeout) based on RTT measurements
- **Option 3:** False – TCP uses cumulative ACKs (next expected byte)

Exercise 5: TCP RTT Estimate

Problem

Suppose that TCP has an RTT estimate that is:

a. Far too small. What is the consequence?

b. Far too large. What is the consequence?

Exercise 5: TCP RTT Estimate - Solution

Solution

a. RTT too small: Many unnecessary timeouts

- Premature retransmissions
- Wasted bandwidth
- Unnecessary congestion control triggers

b. RTT too large: Takes a long time to detect packet loss

- Slow reaction to actual losses
- Reduced throughput
- Poor performance on lossy networks

Exercise 6: Go-Back-N with Reordering

Multiple Choice

Consider the **Go-Back-N protocol**, and assume that the channel can **re-order** messages such that when the n-th packet is sent, it can only be bypassed by the packet that was sent **immediately after it**.

In such a case, the algorithm is correct only if:

1. the sequence number space is at least **one** bigger than the window size
2. the sequence number space is at least **two** bigger than the window size
3. the sequence number space is at least **three** bigger than the window size
4. the sequence number space is at least **four** bigger than the window size
5. None of the above

Go-Back-N: Standard Case Reminder

TCP Recitation Remainder

What is the minimal number of sequence numbers required for Go-Back-N with window size N?

Answer: $N + 1$

Why N+1?

With only N sequence numbers, the receiver cannot distinguish between:

- A new packet with sequence number 0
- A retransmission of an old packet with sequence number 0

Why N+1 Fails with Reordering

Example: $N = 2$, Seq Numbers $\{0, 1, 2\}$ (N+1=3)

Step 1: Sender window $[0, 1]$, sends pkt 0, pkt 1
Step 2: Receiver gets both, sends ACK 1 (cumulative)
Step 3: Sender receives ACK 1 $\rightarrow$ window slides to $[2, 0]$
Step 4: Sender sends pkt 2, then pkt 0 (new!)
Step 5: Packets **reorder**: pkt 0 arrives before pkt 2

The Problem

Receiver is expecting pkt 2, but receives pkt 0.

Is this the **new** pkt 0 (from window $[2, 0]$) that arrived early?
Or is it an **old** pkt 0 (from window $[0, 1]$) that was delayed?

With only N+1 sequence numbers, the receiver cannot tell!

Why N+2 is Sufficient

Example: $N = 2$, Seq Numbers $\{0, 1, 2, 3\}$ (N+2=4)

Step 1: Sender window $[0, 1]$, sends pkt 0, pkt 1
Step 2: Receiver gets both, sends ACK 1
Step 3: Sender receives ACK 1 $\rightarrow$ window slides to $[2, 3]$
Step 4: Sender sends pkt 2, pkt 3
Key: Old packets had seq $\{0, 1\}$, new packets have seq $\{2, 3\}$
No overlap! Even with adjacent reordering, no ambiguity.

Why +2?

- $+1$ separates old window from new window (standard GBN)
- $+1$ extra handles the case where adjacent packets swap
- Total: **N+2** sequence numbers needed

Exercise 6: Solution

Solution

Answer: 2

The sequence number space must be at least **two bigger** than the window size ($N + 2$).

Scenario	**Seq Numbers Needed**
Standard GBN (no reordering)	$N + 1$
GBN + adjacent reordering	$N + 2$

Key Insight

Each "level" of reordering (how far packets can jump) adds one more sequence number needed to avoid ambiguity.

Exercise 7: Sliding Window ACK Ranges

Problem

Consider Sliding Window protocols with **window size = 25** and **sequence numbers 0 to** 2^{32}. At time t, the receiver has ACKed all packets with seq < 100 and hasn't received packet 100.

At time t, receiver gets a packet with sequence number X. For what values of X will the receiver send an ACK?

1. Go-Back-N is used
2. Selective-Repeat is used

Exercise 7: Solution

Solution

1. Go-Back-N: $X \in [0, 2^{32}]$ (any packet)
Go-Back-N always sends a cumulative ACK for the last in-order packet received. Any packet triggers an ACK (even if out of order).

2. Selective-Repeat: $X \in [75, 124]$

- Receiver window: $[100, 124]$ (next 25 expected)
- Also accepts retransmissions: $[75, 99]$ (previous window)
- Combined valid range: $[75, 124]$

Exercise 8: TCP File Transfer

Problem

Consider transferring an enormous file of L bytes from host A to host B. Assume MSS $= 536$ bytes.

a. What is the maximum value of L such that TCP sequence numbers are not exhausted?

b. For this L, how long to transmit? Assume 66 bytes of headers per segment, 100 Mbps link, back-to-back transmission.

Exercise 8: Solution

Part a

Maximum $L = 2^{32}$ bytes
TCP sequence numbers are 32 bits, allowing 2^{32} unique byte positions.

Part b

- Number of segments: $\lceil 2^{32}/536 \rceil \approx 8,012,999$ segments
- Each segment: $536 + 66 = 602$ bytes $= 4816$ bits
- Total bits: $8,012,999 \times 4816 \approx 3.86 \times 10^{10}$ bits
- Time: $\frac{3.86 \times 10^{10}}{100 \times 10^{6}} \approx$ **386 seconds**

Exercise 9: UDP Segment Length

True/False

The theoretical limit for the data length of a UDP segment is less than 64,000 bytes.

Solution

False

The UDP length field is 16 bits, allowing up to $2^{16} - 1 =$ **65,535 bytes** (including 8-byte header).

Maximum data: $65,535 - 8 = 65,527$ bytes

Exercise 10: Go-Back-N Timing

Problem

Go-Back-N with window size 70, sequence numbers 0-2047. RTT is fixed. Transmission delay is d seconds. Sender starts at time t_0.

If **only the first packet is lost**, at what time (earliest) will the receiver be **prepared to accept** all first 70 packets?

Note: Go-Back-N does not buffer out-of-order packets. Packets 1-69 arrive but are discarded. The receiver is "ready to receive" them, once the retransmitted packet 0 arrives.

A. Timer starts **after** sender completes transmitting first packet
B. Timer starts **before** sender starts transmitting first packet

Exercise 10: Solution

Solution

A. Timer starts after transmission:

$$t = t_0 + d + \text{timeout} + \frac{RTT}{2} + d$$

B. Timer starts before transmission:

$$t = t_0 + \text{timeout} + \frac{RTT}{2} + d$$

Exercise 11: HTTP Download

Problem

- HTML page: 10K bits (on server A)
- 5 images: 30K bits each (on server B)
- RTT to A: 500 ms RTT to B: 100 ms
- Both links: 1000 Mbps

Calculate total download time for:

A. Non-persistent HTTP, no parallel connections

B. Non-persistent HTTP, with parallel connections

C. Persistent HTTP, no pipelining

D. Persistent HTTP, with pipelining

HTTP Timing: Formulas

Time Components

- $TCP = RTT$ (connection setup $= 1$ RTT)
- $RTT_A = 500$ ms $\quad RTT_B = 100$ ms
- HTML transmission: $T_A = \frac{10K}{1000M} = 0.01$ ms
- IMG transmission: $T_B = \frac{30K}{1000M} = 0.03$ ms

Object Download Time

For each object: TCP handshake + GET/Response + Transmission

$$= RTT + RTT + T_{trans} = 2 \cdot RTT + T_{trans}$$

Exercise 11: Solution A & B

A. Non-persistent, no parallel

$$T = (2 \cdot RTT_A + T_A) + 5 \times (2 \cdot RTT_B + T_B)$$

$$T = (1000 + 0.01) + 5 \times (200 + 0.03) = 1000.01 + 1000.15 = \textbf{2000.16 ms}$$

B. Non-persistent, parallel

$$T = (2 \cdot RTT_A + T_A) + (2 \cdot RTT_B + 5 \cdot T_B)$$

$$T = 1000.01 + 200.15 = \textbf{1200.16 ms}$$

(All 5 images fetched in parallel, but share link bandwidth)

Exercise 11: Solution C & D

C. Persistent, no pipelining

$$T = (2 \cdot RTT_A + T_A) + RTT_B + 5 \times (RTT_B + T_B)$$

$$T = 1000.01 + 100 + 5 \times 100.03 = 1000.01 + 100 + 500.15 = \textbf{1600.16 ms}$$

(One TCP to B, then sequential GET/response for each image)

D. Persistent, with pipelining

$$T = (2 \cdot RTT_A + T_A) + RTT_B + (RTT_B + 5 \cdot T_B)$$

$$T = 1000.01 + 100 + 100.15 = \textbf{1200.16 ms}$$

(All 5 GETs pipelined, responses stream back sequentially)

HTTP Timing: Summary

HTTP Type	Time (ms)
Non-persistent, sequential	2000.16
Non-persistent, parallel	1200.16
Persistent, no pipelining	1600.16
Persistent, with pipelining	1200.16

Key Insight

- Transmission times included for completeness
- Parallel and pipelining have same total time (bandwidth is the bottleneck)
- Pipelining uses fewer resources (single connection vs 5 connections)

Good Luck on the Exam!

Questions?